

POLITECNICO DI TORINO

Corso di Laurea in Ingegneria Gestionale
Classe L8 – Ingegneria dell'Informazione



**Movie Cast Builder: individuare il miglior cast per una
produzione cinematografica di successo**

Relatore
Prof. Giuseppe Bruno Averta

Candidato
Valle Diego
246880

A.A 2024/2025

Sommario

1	Proposta di progetto.....	1
1.1	Studente proponente.....	1
1.2	Titolo della proposta.....	1
1.3	Descrizione del problema proposto.....	1
1.4	Descrizione della rilevanza gestionale del problema.....	1
1.5	Descrizione dei data-set per la valutazione.....	1
1.6	Descrizione preliminare degli algoritmi coinvolti.....	2
1.7	Descrizione preliminare delle funzionalità previste per l'applicazione software.....	3
2	Descrizione del problema affrontato.....	4
3	Descrizione del data-set utilizzato per l'analisi.....	5
3.1	Introduzione.....	5
3.2	Tabella movies.....	5
3.3	Tabella actor.....	6
4	Descrizione ad alto livello delle strutture dati e degli algoritmi utilizzati.....	7
4.1	Descrizione delle strutture dati: i packages e le classi.....	7
4.1.1	it.polito.tdp.movieCastBuilder.....	7
4.1.2	it.polito.tdp.movieCastBuilder.db.....	7
4.1.3	it.polito.tdp.movieCastBuilder.model.....	8
4.2	Descrizione degli algoritmi utilizzati.....	8
4.2.1	Algoritmi di ricerca.....	8
4.2.2	Ricorsione.....	12
4.2.3	Algoritmi per mostrare o modificare l'output.....	16
5	Interfaccia e video dimostrativo.....	18
5.1	Sezione Genera Cast.....	18
5.2	Sezione Best Cast.....	18
5.3	Video dimostrativo.....	19
6	Risultati sperimentali e esempi.....	20
6.1	Esempio 1.....	20
6.3	Esempio 2.....	21
6.2	Esempio 3.....	22
7	Valutazioni sui risultati ottenuti.....	23

1 Proposta di progetto

1.1 Studente proponente

s246880 Valle Diego

1.2 Titolo della proposta

Movie Cast Builder: individuare il miglior cast per una produzione cinematografica di successo

1.3 Descrizione del problema proposto

Suggerire a una casa cinematografica il miglior cast per una nuova produzione, combinando una selezione di artisti. La prioritizzazione avviene partendo da un dataset storico che valuta il successo di grandi film in termini di incassi e valutazioni della critica. L'obiettivo del software è di simulare diversi scenari variando 6 parametri (Incasso del film, approvazione della critica, approvazione del pubblico, sintonia tra gli attori, intesa con il regista, affinità degli attori con il genere di film) al fine di ingaggiare il miglior team possibile e massimizzare incassi, soddisfazione degli attori e in generale il successo del film.

1.4 Descrizione della rilevanza gestionale del problema

Creare un cast cinematografico efficace è complesso perché richiede l'equilibrio tra talento, chimica tra attori, disponibilità, budget e tanti altri fattori spesso trascurati. Un'applicazione che incrocia dati su attori come esperienza, ruoli passati o valutazioni, può semplificare il processo decisionale e proporre diversi scenari per individuare soluzioni e testarne la qualità in modo artificiale. I benefici gestionali sono risparmio di tempo, riduzione degli errori di casting e maggiore coerenza artistica. Inoltre, consente ai produttori di prevedere impatti commerciali in termini di ricavi e rating, migliorando la pianificazione del progetto e ponendo solide basi per le fasi successive di produzione del film.

1.5 Descrizione dei data-set per la valutazione

Il dataset selezionato per il progetto è "Top 1000 Movies by IMDB Rating", reperibile su Kaggle con licenza di dominio pubblico. Il database è una raccolta completa di informazioni su 1000 film selezionati in base alla valutazione IMDB. Ogni riga rappresenta un film, e il dataset è strutturato su 16 attributi, descritti brevemente di seguito:

- **Poster_Link** – URL dell'immagine del poster del film pubblicata su IMDB.
- **Series_Title** – Nome del film.
- **Released_Year** – Anno in cui il film è stato distribuito per la prima volta
- **Certificate** – Classificazione di età (es. PG-13, R) secondo il sistema di rating americano.
- **Runtime** – Durata complessiva del film espressa in minuti.

- **Genre** – Categorie di genere cinematografico associate al film (es. Dramma, Azione).
- **IMDB_Rating** – Valutazione media del film espressa in decimali, su una scala da 1 a 10.
- **Overview** – Sinossi del film.
- **Meta-score** – Punteggio assegnato al film basato su recensioni critiche espressa in numeri interi, su una scala da 0 a 100
- **Director**– Nome del regista del film.
- **Star1** – Primo attore/attrice protagonista del film.
- **Star2** – Secondo attore/attrice protagonista del film.
- **Star3** – Terzo attore/attrice protagonista del film.
- **Star4** – Quarto attore/attrice protagonista del film.
- **No_of_Votes** – Quantità di recensioni critiche considerate per calcolare il meta-score.
- **Gross**– Incasso del film, espresso in dollari.

Verranno fatte valutazioni per integrare altre informazioni mancanti o aggiuntive dal datasetl “MDb Actors and Movies”, anch’esso reperibile su Kaggle con licenza di dominio pubblico.

1.6 Descrizione preliminare degli algoritmi coinvolti

L’algoritmo principale è una ricorsione con lo scopo ottimizzare un formula predittiva di successo. La compagnia sceglie il genere e il regista, e assegna un peso a 5 parametri chiave, sulla base dei quali il sistema suggerisce la combinazione di 4 attori con il punteggio più alto per il progetto. Gli attori considerati nella ricorsione sono solamente gli attori che hanno recitato almeno una volta nel genere scelto.

Prima che parta la ricorsione è necessario normalizzare i valori delle 5 caratteristiche, affinché possano essere comparate tra di loro equamente. A questo scopo verrà creata una mappa che associa ad ogni attore 4 valori, legati alle 4 delle 5 caratteristiche che vengono considerate. Si individuerà poi il valore massimo di queste statistiche tramite algoritmi di iterazione, al fine di normalizzarle a valori che vanno da 0 a 1. Lo stesso verrà fatto con la quinta caratteristica, ovvero la sintonia fra attori. Per questa caratteristica viene creato un grafo non orientato pesato dove i nodi sono gli attori, gli archi esistono se questi attori hanno recitato almeno una volta insieme e il peso rappresenta il numero di film che condividono. Al fine di trovare il massimo di questa caratteristica, sarà necessaria una ricorsione.

Quindi parte la ricorsione principale, che controlla tutte le possibili combinazioni di squadre da 4 star, normalizza per ciascuna combinazione le 5 caratteristiche, applica le percentuali di interesse inserite dall’utente e assegna dunque un voto alla combinazione sommando le 5 caratteristiche. A fine ricorsione viene individuato il cast che ha ottenuto il punteggio più alto.

Saranno inoltre presenti algoritmi più semplici di ricerca record per creare una tabella con lo storico dei film a cui hanno preso parte gli attori del cast, e una ricorsione più semplice per permettere all’utente eventualmente di sostituire un attore che non vuole contattare per la realizzazione della pellicola.

1.7 Descrizione preliminare delle funzionalità previste per l'applicazione software

L'applicazione prevede una sezione iniziale dedicata all'immissione degli input. All'inizio è selezionabile solo una tendina con tutti i generi cinematografici, che consente all'utente di sceglierne uno. Dopodichè, si attiva una sezione in cui una tendina permetterà all'utente di scegliere un regista, consentendo la scelta solamente di registi che hanno già fatto almeno un film col genere scelto. A questo punto sarà possibile selezionare il valore delle 5 caratteristiche. Questi parametri possono assumere valori numerici da 1 a 5, e indicano l'interesse dell'utente per capire quali caratteristiche tenere maggiormente conto nell'algoritmo.

Selezionato tutti i valori parte la ricorsione che fornisce in una finestra dedicata all'output il risultato, ovvero il cast composto dalle 4 star con la maggiore probabilità di successo per la pellicola da produrre. A questo punto è possibile visionare lo storico dei film di queste star, tramite una tabella contenente tutti i film in cui hanno recitato e qualche informazione base su questi film.

Infine, nel caso che la casa cinematografica fosse contraria a contattare un determinato attore è possibile escluderlo dalla lista del cast, e in questo caso parte una ricorsione molto più breve che trova il sostituto più adatto.

2 Descrizione del problema affrontato

Produrre un film di successo per una casa cinematografica non è un'impresa semplice: sono molti i fattori da considerare. Tra i più rilevanti vi è la capacità di individuare un cast vincente, capace di conquistare il pubblico e la critica, e di risultare coerente con la visione e le esigenze del film che si intende realizzare.

Questo software mira a semplificare questo processo fornendo alla casa cinematografica indicazioni su un cast potenzialmente vincente, in grado di rispondere in modo efficace alle sue esigenze produttive e artistiche. Per raggiungere questo obiettivo, il programma analizza i 1000 film con i punteggi IMDb più alti fino al 2024, assegnando a ciascun **cast** (composto da ogni possibile combinazione di 4 attori) **6 caratteristiche**.

Le prime 5, vengono calcolate come la media delle stesse caratteristiche associate ai singoli attori, e sono le seguenti:

- **Incasso del film** – Media dei ricavi dei film in cui ha partecipato l'attore
- **Approvazione della critica** – Media dei meta score dei film in cui ha partecipato l'attore
- **Approvazione del pubblico** – $\beta * (\text{Media dei imdb Rating dei film in cui ha partecipato l'attore}) + (1 - \beta) * (\text{numero di voti})$
- **Affinità con il genere scelto** – $\alpha * (\text{Percentuale di film interpretati dall'attore appartenenti al genere scelto sul totale della sua filmografia}) + (1 - \alpha) * (\text{Numero di film appartenenti al genere scelto in cui ha recitato})$.
- **Intesa con il regista** – $\alpha * (\text{Percentuale di film condivisi tra attore e regista rispetto al totale dei film interpretati dall'attore}) + (1 - \alpha) * (\text{Numero totale dei film realizzati insieme al regista})$.

La sesta caratteristica invece è associata al cast, ed è la **sintonia tra gli attori**: essa si riferisce a un cast di quattro interpreti e valuta il grado di collaborazione pregressa, basandosi sul numero di film in cui hanno recitato insieme.

3 Descrizione del data-set utilizzato per l'analisi

3.1 Introduzione

Per creare il database utilizzato dal software sono stati utilizzati 2 dataset forniti dal sito Kaggle, entrambi dotati di licenza di dominio pubblico.

È stato dunque creato un nuovo database chiamato **topfilm** nell'ambiente di HeidiSQL, contenente 2 tabelle. Dal momento che i dati erano forniti in formato CSV, è stata svolta una conversione in formato SQL attraverso il comando **import CSV**.



3.2 Tabella movies

La tabella è una raccolta completa di informazioni su 1000 film selezionati in base alla più alta valutazione IMDB. Ogni riga rappresenta un film e il dataset è strutturato su 16 attributi, descritti brevemente di seguito:

- **Poster_Link** – URL dell'immagine del poster del film pubblicata su IMDB.
- **Series_Title** – Nome completo del film come riportato su IMDB.
- **Released_Year** – Anno in cui il film è stato distribuito per la prima volta.
- **Certificate** – Classificazione di età (es. PG-13, R) secondo il sistema di rating americano.
- **Runtime** – Durata complessiva del film espressa in minuti.
- **Genre** – Categorie di genere associate al film (es. Dramma, Azione).
- **IMDB_Rating** – Valutazione media del film espressa in decimali, su una scala da 1 a 10.
- **Overview** – Trama sintetica del film, generalmente una breve descrizione narrativa.
- **Meta_score** – Punteggio assegnato al film da Metacritic, basato su recensioni critiche.
- **Director** – Nome del regista principale del film.
- **Star 1** – Primo attore/attrice protagonista del film.
- **Star 2** – Secondo attore/attrice protagonista del film.
- **Star 3** – Terzo attore/attrice protagonista del film.
- **Star 4** – Quarto attore/attrice protagonista del film.
- **No_of_Votes** – Quantità di recensioni critiche considerate per calcolare il meta-score.
- **Gross** – Incasso globale stimato del film, generalmente espresso in dollari.

Alcuni film presentavano valori mancanti nei campi **Gross** e **Meta_score**. Per evitare di escluderli dall'analisi, è stato deciso di assegnare a questi campi il valore minimo registrato nella rispettiva categoria.

#	Nome	Tipo di dati
1	Poster_Link	VARCHAR
2	Series_Title	MEDIUMTEXT
3	Released_Year	INT
4	Certificate	VARCHAR
5	Runtime	VARCHAR
6	Genre	LONGTEXT
7	IMDB_Rating	DOUBLE
8	Overview	LONGTEXT
9	Director	VARCHAR
10	Meta_score	INT
11	Star1	VARCHAR
12	Star2	VARCHAR
13	Star3	VARCHAR
14	Star4	VARCHAR
15	No_of_Votes	INT
16	Gross	INT

3.3 Tabella actor

Questa tabella contiene informazioni di base sugli attori, utili innanzitutto per evitare di selezionare attori deceduti e per gestire eventuali casi di omonimia e individuare correttamente quale attore con lo stesso nome sia quello giusto.

Al fine di non affaticare l'applicazione con eccessivi calcoli si è preferito snellire il database eliminando dati errati e/o ridondanti, nonché oggetti inutili ai fini del software.

La tabella è composta da 2703 elementi ed è definita da 5 attributi, descritti brevemente di seguito:

- **primaryName** – Nome e cognome della persona.
- **birthYear** – Data di nascita.
- **deathYear** – Data della morte (Il campo è vuoto se l'attore è vivo).
- **primaryProfession**– Elenco delle sue competenze in ambito lavorativo.
- **knownForTitle** – Titolo del film per cui è più conosciuto.

#	Nome	Tipo di dati
1	primaryName	VARCHAR
2	birthYear	INT
3	deathYear	INT
4	primaryProfession	LONGTEXT
5	knownForTitle	VARCHAR

4 Descrizione ad alto livello delle strutture dati e degli algoritmi utilizzati

L'applicazione è stata sviluppata in linguaggio Java seguendo il **pattern MVC** (Model-View-Controller) ed il **pattern DAO** (Data-Access-Object) tramite i quali sono stati separati l'accesso al database, i processi algoritmici per la manipolazione dei dati e l'interfaccia utente.

4.1 Descrizione delle strutture dati: i packages e le classi

Grazie all'utilizzo dei package, le tre fasi del progetto vengono mantenute separate: una dedicata al **model**, una al **controller** e una alla **gestione del database**.

```

  ▾ M.J. > movieCastBuilder [movieCastBuilder master]
    ▾ > src/main/java
      ▾ > it.polito.tdp.movieCastBuilder
        > > EntryPoint.java
        > > FXMLController.java
        > > Main.java
      ▾ > it.polito.tdp.movieCastBuilder.db
        > > DBConnect.java
        > > imdbDAO.java
      ▾ > it.polito.tdp.movieCastBuilder.model
        > > Actor.java
        > > Cast.java
        > > Model.java
        > > Movie.java
        > > Pair.java

```

4.1.1 it.polito.tdp.movieCastBuilder

È il package che si occupa della procedura di avvio dell'applicazione e gestisce l'interfaccia dell'utente. Di seguito le sue tre classi:

- **EntryPoint** – Necessario per l'avvio dell'applicazione.
- **Main** – Gestisce l'avvio dell'interfaccia grafica.
- **FXMLController** – Gestisce l'interfaccia grafica dell'applicazione. Riceve i dati in input dall'utente, li interpreta e li inoltra al package responsabile della logica applicativa tramite l'oggetto Model. Successivamente, riceve i risultati dal livello logico e li visualizza come output nell'interfaccia utente.

4.1.2 it.polito.tdp.movieCastBuilder.db

È il package che consente la connessione alla base di dati e la interroga attraverso query scritte in linguaggio SQL. Di seguito le sue due classi:

- **DBConnect** – Classe che permette la connessione al database tramite pooling e che permette di risparmiare tempo di apertura e di chiusura di una connessione grazie alla classe **HikariDataSource**.

- **imdbDAO**– Classe che riceve parametri dal Model e interroga il database tramite query SQL. Crea delle strutture dati idonee all'organizzazione dei dati raccolti e li restituisce al Model.

4.1.3 it.polito.tdp.movieCastBuilder.model

Il package contiene la logica applicativa e svolge la maggior parte delle operazioni e dei calcoli. Di seguito le sue 5 classi:

- **Actor** – Definisce un attore. I suoi attributi comprendono i dati estratti dalla tabella "actor" oltre ad altri attributi aggiuntivi necessari per calcolare le caratteristiche utili alla selezione del cast migliore.
- **Cast** – Definisce un cast. I suoi attributi comprendono 4 oggetti Actor e altri attributi necessari al fine di individuare il miglior cast possibile.
- **Model** – Rappresenta la classe principale del progetto, all'interno della quale sono contenuti tutti i dati e le strutture necessarie per il corretto svolgimento dei processi. Il Model gestisce la connessione sia con il database sia con l'interfaccia utente.
- **Movie** – Definisce un film. I suoi attributi sono i dati che vengono estratti dalla tabella "movies".
- **Pair**– Definisce una coppia di attori. I suoi attributi sono i 2 oggetti Actor, insieme al numero di film in cui hanno recitato insieme.

4.2 Descrizione degli algoritmi utilizzati

4.2.1 Algoritmi di ricerca

In questa sezione l'utente sceglie dalla ComboBox **cmbGenre** un genere cinematografico, precedentemente riempita grazie al metodo **getGenre()**. Una volta selezionato, il metodo **getDirector()** riempie la **cmbDirector** con tutti i registi che hanno partecipato ad almeno un film con il genere scelto. Entrambi i metodi interrogano allo stesso modo il dataset per ottenere i dati, con le loro rispettive query.

```
public List<String> getDirector(String genre) {
    // TODO Auto-generated method stub
    String sql = "SELECT DISTINCT m.director\r "
        + "FROM movies m\r "
        + "JOIN actor a ON m.director = a.primaryName\r "
        + "WHERE FIND_IN_SET(?, REPLACE(m.genre, ' ', ' ', ',')) \r "
        + "AND a.deathYear = 0\r "
        + "ORDER BY m.director;\r "
        + "\r "
        + "";
    List<String> lDirector = new ArrayList<>();
    try {
        Connection conn = DBConnect.getConnection();
        PreparedStatement st = conn.prepareStatement(sql);
        st.setString(1, genre);

        ResultSet res = st.executeQuery();
        while(res.next()) {
            lDirector.add(res.getString("director"));
        }
        conn.close();
    } catch (SQLException e) {
        e.printStackTrace();
        return null;
    }
    return lDirector;
}
```

Successivamente, l'utente seleziona i parametri di interesse grazie a sei slider, e farà poi partire la ricerca del cast. In qualsiasi momento sarà possibile ricominciare da capo la selezione dei parametri grazie al tasto **Reset**.

Viene quindi creato il grafo con il metodo **creaGrafo()**, utile a calcolare le caratteristiche associate al cast. Il metodo **getVertici()** otterrà i vertici, ovvero tutti gli attori ancora in vita che hanno partecipato almeno una volta ad un film con il genere cinematografico selezionato. Il metodo **getArchi()** otterrà invece gli archi, che rappresentano il conto dei film che 2 attori hanno condiviso.

```
private void creaGrafo(String genere) {
    // TODO Auto-generated method stub
    this.grafo = new SimpleWeightedGraph<>(DefaultWeightedEdge.class);

    //aggiungo i vertici
    Graphs.addAllVertices(this.grafo, this.dao.getVertici(genre));
    System.out.println("Attori: " + this.grafo.vertexSet().size());

    //aggiungo archi
    for(Pair p : this.dao.getArchi(this.grafo.vertexSet())) {
        Graphs.addEdgeWithVertices(this.grafo, p.getA1(), p.getA2(), p.getW());
    }
    System.out.println("Archi: " + this.grafo.edgeSet().size());
}
```

Parte del metodo getArchi()

```
public List<Pair> getArchi(Set<Actor> lActor) {
    // TODO Auto-generated method stub
    String sql = "SELECT \r "
        + "    LEAST(actor1, actor2) AS a1,\r "
        + "    GREATEST(actor1, actor2) AS a2,\r "
        + "    COUNT(*) AS weight\r "
        + "FROM (\r "
        + "    SELECT genre, star1 AS actor1, star2 AS actor2 FROM movies \r "
        + "    UNION ALL\r "
        + "    SELECT genre, star1 AS actor1, star3 AS actor2 FROM movies \r "
        + "    UNION ALL\r "
        + "    SELECT genre, star1 AS actor1, star4 AS actor2 FROM movies \r "
        + "    UNION ALL\r "
        + "    SELECT genre, star2 AS actor1, star3 AS actor2 FROM movies \r "
        + "    UNION ALL\r "
        + "    SELECT genre, star2 AS actor1, star4 AS actor2 FROM movies \r "
        + "    UNION ALL\r "
        + "    SELECT genre, star3 AS actor1, star4 AS actor2 FROM movies \r "
        + ") AS actor_pairs\r "
        + "WHERE actor1 <> actor2\r "
        + "GROUP BY actor1, actor2\r "
        + "ORDER BY weight DESC;\r "
        + "";
```

Interessante notare che con l'estrapolazione dei dati sugli attori dal dataset, è stato necessario gestire il problema degli omonimi. Non avendo abbastanza informazioni per affermare immediatamente quale dei 2 omonimi fosse l'attore ad aver partecipato al film, sono stati escogitati stratagemmi per individuare quello giusto. In ordine, viene controllato:

- Se l'attributo associato all'attore **KnowForTitle** corrisponda al titolo del film.

- Se non è identico si controlla se è simile, utilizzando la classe **JaroWinkler** presente nella libreria `java-string.similarity`, che permette di riconoscere quanto le stringhe si assomigliano tramite un valore numerico. La soglia di somiglianza scelta dopo aver effettuato dei test è 0,62 (Questo passaggio aiuta a riconoscere l'attore giusto quando si parla di saghe, come per esempio Harry Potter o Il signore degli anelli)
- In assenza di somiglianze, viene scelto chi tra i 2 omonimi contiene "actor" o "actress" nell'attributo **primaryProfession**.

Se nessuno di questi passaggi riesce a decretare l'attore corretto, purtroppo non c'è modo di sapere chi dei due sia l'attore giusto. Bisogna comunque sottolineare che nel caso venisse scelto l'attore sbagliato, in nessun modo cambierebbe l'esito della scelta del miglior cast possibile, semplicemente verrebbero mostrate le info dell'attore sbagliato nell'output. Comunque testando non è stato riscontrando alcun caso in cui i 3 stratagemmi sopra descritti non siano riusciti a trovare l'attore giusto.

Un secondo problema è stato riscontrato con alcuni attori viventi associati a film usciti prima della loro nascita, a causa della presenza di omonimi deceduti. Il problema è stato risolto introducendo un controllo aggiuntivo, che considera validi solo i film in cui l'attore aveva almeno sei anni al momento dell'uscita.

```
public List<Actor> getVertici(String genre) {
    // TODO Auto-generated method stub

    String sql = "WITH attori AS (\r "
        + "    SELECT Star1 AS attore, series_Title, Released_Year\r "
        + "    FROM topFilm.movies\r "
        + "    WHERE Genre LIKE ?\r "
        + "    UNION\r "
        + "    SELECT Star2, series_Title, Released_Year\r "
        + "    FROM topFilm.movies\r "
        + "    WHERE Genre LIKE ?\r "
        + "    UNION\r "
        + "    SELECT Star3, series_Title, Released_Year\r "
        + "    FROM topFilm.movies\r "
        + "    WHERE Genre LIKE ?\r "
        + "    UNION\r "
        + "    SELECT Star4, series_Title, Released_Year\r "
        + "    FROM topFilm.movies\r "
        + "    WHERE Genre LIKE ?\r "
        + ")\r "
        + "SELECT\r "
        + "    af.attore AS name,\r "
        + "    a.birthYear,\r "
        + "    a.primaryProfession AS profession,\r "
        + "    a.knownForTitle AS knowForTitle,\r "
        + "    af.series_Title AS movie,\r "
        + "    af.Released_Year \r "
        + "FROM\r "
        + "    attori af\r "
        + "JOIN\r "
        + "    actor a ON af.attore = a.primaryName\r "
        + "WHERE\r "
        + "    a.deathYear = 0 \r "
        + "    AND a.birthYear + 6 <= af.Released_Year \r "
        + "ORDER BY\r "
        + "    a.primaryName, af.series_Title;";
```

```

Map<String,Actor> mActor = new TreeMap<>();
try {
    Connection conn = DBConnect.getConnection() ;
    PreparedStatement st = conn.prepareStatement(sql) ;
    st.setString(1, "%" + genre + "%");
    st.setString(2, "%" + genre + "%");
    st.setString(3, "%" + genre + "%");
    st.setString(4, "%" + genre + "%");

    ResultSet res = st.executeQuery() ;
    while(res.next()) { // è necessario evitare omonimi, non avendo modo di sapere qual'è
        //l'attore giusto, ho escogitato questi metodi per scegliere quello verosimilmente corretto

        Jarowinkler jw = new Jarowinkler();
        double now = jw.similarity(res.getString("knowForTitle"), res.getString("movie"));

        Actor a = new Actor(res.getString("name"),res.getInt("birthYear"),res.getString("profession"),
            res.getString("knowForTitle"),now,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0);

        if(!mActor.containsKey(res.getString("name"))) { //se il nome non c'è lo aggiungo, altrimenti
            //deciderò quale dei 2 attori tenere

            a.setJw(now);
            mActor.put(res.getString("name"), a);

        }else if((mActor.get(res.getString("name")).getBirthYear() != res.getInt("birthYear")) ||
            (!mActor.get(res.getString("name")).getProfession().equals(res.getString("profession"))))
            { //controllo che non sia lo stesso attore

                double old = mActor.get(res.getString("name")).getJw();

                if(now >= 0.62 && old < 0.62) { //Controllo se il nuovo attore entra per somiglianza nome

                    mActor.remove(res.getString("name"));
                    mActor.put(res.getString("name"), a);

                }else if(!(now < 0.62 && old >= 0.62)){ // Controllo se rimane il vecchio attore per somiglianza nome

                    if(((a.getProfession().contains("actor")) && !(mActor.get(res.getString("name")).getProfession()
                        .contains("actor")))) || ((a.getProfession().contains("actress")) && !(mActor.get(res.
                            getString("name")).getProfession().contains("actress")))) { // controllo se il nuovo
                        //attore ha actor nel nome, a differenza di quello vecchio

                            a.setJw(now);
                            mActor.put(res.getString("name"), a);

                        }else if(!(((a.getProfession().contains("actor")) && (mActor.get(res.getString("name")).
                            getProfession().contains("actor")) || (!(a.getProfession().contains("actress")) && (mActor.
                                get(res.getString("name")).getProfession().contains("actress"))))) { // controllo se
                            //il vecchio attore ha actor nel nome, a differenza di quello nuovo

                                if(now > old) { //non è sicuro chi sia l'attore giusto.
                                    //a caso scelgo di mettere chi ha il nome più simile.

                                        mActor.remove(res.getString("name"));
                                        mActor.put(res.getString("name"), a);

                                    }

                                }

                            }else if(a.getJw() < now) { //Aggiorno il valore jw
                                a.setJw(now);
                            }

                        }

                    conn.close();
                } catch (SQLException e) {
                    e.printStackTrace();
                    return null;
                }

            }

        return new ArrayList<>(mActor.values());
    }
}

```

4.2.2 Ricorsione

Il cuore del software consiste nell'individuare il cast con il massimo valore dell'attributo **statCast**, esplorando ogni possibile combinazione tramite un algoritmo ricorsivo. Per preparare i dati necessari, il metodo **findCast()** comincia calcolando le caratteristiche necessarie collegate ad ogni singolo attore grazie ad un doppio ciclo for, aggiornando di volta in volta i valori massimi e minimi rilevanti.

```
public Cast findCast(String genre, String director, double inputIncassoFilm, double inputApprovazioneCritica,
    double inputApprovazionePubblico, double inputSintoniaAttori, double inputIntesaRegista,
    double inputAffinitaGenere) {

    // TODO Auto-generated method stub
    creaGrafo(genre);

    //Calcolo le statistiche con un doppio for annodato
    for(Actor a : this.grafo.vertexSet()) {
        for(Movie m : lMovies) {

            if((m.getCast()).contains(a.getName())) {

                if(m.getMetaScore() == 0)
                    m.setMetaScore(minMetaScore);
                if(m.getGross() == 0)
                    m.setGross(minGross);

                a.setnMoviesTot(a.getnMoviesTot() + 1);
                a.setGrossTot(a.getGrossTot() + m.getGross());
                a.setMetaScoreTot(a.getMetaScoreTot() + m.getMetaScore());
                a.setImbdRatingTot(a.getImbdRatingTot() + ema(beta,normalization(m.getImbdRating(),
                    maxIMDBrating,minIMDBrating),normalization(m.getNumVotes(),maxVotes,minVotes)));
                if(m.getDirector().equals(director)) {
                    a.setnMoviesDirector(a.getnMoviesDirector() + 1);
                }
                if(m.getGenre().contains(genre)) {
                    a.setnMoviesGenre(a.getnMoviesGenre() + 1);
                }
            }
        }
    }

    //Trovo i max e min per Genre e Director
    int maxDirector = 0;
    int maxGenre = 0;
    int minDirector = 1001;//valori scelti altissimi per inizializzare la variabile
    int minGenre = 1001;

    for(Actor a : this.grafo.vertexSet()) {
        if(a.getnMoviesDirector() > maxDirector) {
            maxDirector = (int) a.getnMoviesDirector();
        }else if(a.getnMoviesDirector() < minDirector) {
            minDirector = (int) a.getnMoviesDirector();
        }
        if(a.getnMoviesGenre() > maxGenre) {
            maxGenre = (int) a.getnMoviesGenre();
        }else if(a.getnMoviesGenre() < minGenre) {
            minGenre = (int) a.getnMoviesGenre();
        }
    }

    if(maxDirector == 0) {//caso in cui regista ha un film con attori morti o assenti dal database
        maxDirector = 1;
    }
}
```

```
// Calcolo le caratteristiche

for(Actor a : this.grafo.vertexSet()) {

    a.setIntesaRegista(ema(alfa,a.getnMoviesDirector()/a.getnMoviesTot(),normalization
        (a.getnMoviesDirector(),maxDirector,minDirector)));
    a.setAffinitaGenere(ema(alfa,a.getnMoviesGenere()/a.getnMoviesTot(),normalization
        (a.getnMoviesGenere(),maxGenere,minGenere)));
    a.setIncassoFilm(a.getGrossTot()/a.getnMoviesTot());
    a.setApprovazioneCritica(a.getMetaScoreTot()/a.getnMoviesTot());
    a.setApprovazionePubblico(a.getImbdRatingTot()/a.getnMoviesTot());
}
```

Di seguito, le funzioni sopra citate

```
private double normalization(double v, double vMax, double vMin) {
    // TODO Auto-generated method stub
    double result = 0;

    result = (v - vMin)/(vMax - vMin);

    return result;
}

private double ema(double peso, double v1, double v2) { // Exponential Moving Average
    // TODO Auto-generated method stub
    double result = 0;

    result = peso * v1 + (1 - peso) * v2;

    return result;
}
```

Dopodiché questi valori vengono normalizzati, per riuscire poi a confrontarli tra di loro in maniera equa. Viene anche considerato il livello di importanza che l'utente ha attribuito ad ogni caratteristica.

```
//Normalizzo le caratteristiche e applico le preferenze dell'utente

for (Actor a : this.grafo.vertexSet()) {
    a.setIncassoFilm(normalization(a.getIncassoFilm(), maxIncassoFilm, minIncassoFilm) * inputIncassoFilm);
    a.setApprovazioneCritica(normalization(a.getApprovazioneCritica(), maxApprovazioneCritica,
        minApprovazioneCritica) * inputApprovazioneCritica);
    a.setApprovazionePubblico(normalization(a.getApprovazionePubblico(), maxApprovazionePubblico,
        minApprovazionePubblico) * inputApprovazionePubblico);
    a.setIntesaRegista(normalization(a.getIntesaRegista(), maxIntesaRegista, minIntesaRegista) *
        inputIntesaRegista);
    a.setAffinitaGenere(normalization(a.getAffinitaGenere(), maxAffinitaGenere, minAffinitaGenere) *
        inputAffinitaGenere);
    a.setStatActor(a.getIncassoFilm() + a.getApprovazioneCritica() + a.getApprovazionePubblico() +
        a.getIntesaRegista() + a.getAffinitaGenere());
}
```

Per evitare tempi di esecuzione eccessivamente lunghi, è stato ritenuto opportuno analizzare i cast formati solamente dai 180 attori più promettenti (valore indicato come **limitActors**), ovvero quelli con l'attributo **statActor** più alto. Sono stati effettuati test sia per scegliere questo numero, sia per assicurarsi che la qualità dell'output fosse identica a quella che avrebbe avuto analizzando tutti gli attori disponibili. Per selezionare gli attori più promettenti è stata usata la classe **PriorityQueue**, che ha permesso di ordinare la lista in base al valore di **statActor**.

```
//Seleziono i "limitActors" attori migliori
poolActors.clear();

if(this.grafo.vertexSet().size() > limitActors) {
    for(Actor a : this.grafo.vertexSet()) {

        if(poolActors.size() == limitActors ) {
            if(a.getStatActor() > pq.peek().getStatActor()) {
                Actor r = pq.poll();
                poolActors.remove(r);

                poolActors.add(a);
                pq.add(a);
            }

        }else {
            poolActors.add(a);
            pq.add(a);
        }
    }
}else {
    poolActors.addAll(this.grafo.vertexSet());
}

removed.clear();
```

A questo punto, prima di procedere all'individuazione del nuovo cast, è necessario calcolare i valori massimi e minimi della sesta caratteristica, ovvero la sintonia tra gli attori, operazione che richiede l'uso della prima ricorsione.

Il metodo **cerca()** esplora tutte le possibili combinazioni di 4 attori, calcolando il valore di **sintoniaAttori** come somma di tutti gli archi presenti nel grafo che collegano i 4 interpreti selezionati. Nel calcolo del valore di **sintoniaAttori**, è stato adottato un approccio incrementale, in modo di evitare di ripetere più volte gli stessi calcoli, ottimizzando così le prestazioni del metodo


```

private void cerca(Cast c, List<Actor> rimanenti, int start) {
    // TODO Auto-generated method stub

    //condizione di terminazione

    if(c.getA4() != null) {

        //controllo se sono i risultati min o max

        if(c.getSintoniaAttori() < minSintoniaAttori) {
            minSintoniaAttori = c.getSintoniaAttori();
        } else if(c.getSintoniaAttori() > maxSintoniaAttori) {
            maxSintoniaAttori = c.getSintoniaAttori();
            System.out.println(c.toString());
        }
        return;
    }

    } else if(c.getA3() != null) {

        for(int i = start; i < rimanenti.size(); i++) {

            c.setA4(rimanenti.get(i));
            double backup = c.getSintoniaAttori();

            if(this.grafo.getEdge(c.getA1(), c.getA4()) != null) {
                c.setSintoniaAttori(c.getSintoniaAttori() + this.grafo.getEdgeWeight(this.grafo.getEdge(c.getA1(), c.getA4())));
            }
            if(this.grafo.getEdge(c.getA4(), c.getA2()) != null) {
                c.setSintoniaAttori(c.getSintoniaAttori() + this.grafo.getEdgeWeight(this.grafo.getEdge(c.getA4(), c.getA2())));
            }
            if(this.grafo.getEdge(c.getA3(), c.getA4()) != null) {
                c.setSintoniaAttori(c.getSintoniaAttori() + this.grafo.getEdgeWeight(this.grafo.getEdge(c.getA3(), c.getA4())));
            }
            cerca(c, rimanenti, i + 1);
            c.setA4(null);
            c.setSintoniaAttori(backup);
        }
    }

    } else if(c.getA2() != null) {

        for(int i = start; i < rimanenti.size(); i++) {

            c.setA3(rimanenti.get(i));
            double backup = c.getSintoniaAttori();

            if(this.grafo.getEdge(c.getA1(), c.getA3()) != null) {
                c.setSintoniaAttori(c.getSintoniaAttori() + this.grafo.getEdgeWeight(this.grafo.getEdge(c.getA1(), c.getA3())));
            }
            if(this.grafo.getEdge(c.getA3(), c.getA2()) != null) {
                c.setSintoniaAttori(c.getSintoniaAttori() + this.grafo.getEdgeWeight(this.grafo.getEdge(c.getA3(), c.getA2())));
            }
            cerca(c, rimanenti, i + 1);
            c.setA3(null);
            c.setSintoniaAttori(backup);
        }
    }

    } else if(c.getA1() != null) {

        for(int i = start; i < rimanenti.size(); i++) {

            c.setA2(rimanenti.get(i));
            double backup = c.getSintoniaAttori();

            if(this.grafo.getEdge(c.getA1(), c.getA2()) != null) {
                c.setSintoniaAttori(c.getSintoniaAttori() + this.grafo.getEdgeWeight(this.grafo.getEdge(c.getA1(), c.getA2())));
            }
            cerca(c, rimanenti, i + 1);
            c.setA2(null);
            c.setSintoniaAttori(backup);
        }
    }

    } else {

        for(int i = start; i < rimanenti.size(); i++) {

            c.setA1(rimanenti.get(i));
            cerca(c, rimanenti, i + 1);
            c.setA1(null);
        }
    }
}

```

A questo punto, tutto è pronto per individuare il cast ottimale! Il metodo **cercaCast()** esplora le stesse combinazioni di 4 attori considerate nella ricorsione precedente. Tuttavia, questa volta, per ogni cast formato, il metodo:

- Normalizza il valore di **sintoniaAttori**
- Calcola le altre cinque caratteristiche del cast come media dei valori precedentemente calcolati per ciascun attore

- Somma tutti questi valori per ottenere la **statCast**.

Al termine della ricorsione, il cast con il valore massimo di **statCast** viene restituito come output del programma: si tratta del cast con la maggior probabilità di successo.

```
private Cast cercaCast(Cast c, List<Actor> rimanenti, int start, double inputSintoniaAttori) {
    // TODO Auto-generated method stub

    //condizione di terminazione

    if(c.getA4() != null) {

        c.setIncassoFilm((c.getA1().getIncassoFilm() + c.getA2().getIncassoFilm() + c.getA3().getIncassoFilm()
            + c.getA4().getIncassoFilm())/4);
        c.setApprovazioneCritica((c.getA1().getApprovazioneCritica() + c.getA2().getApprovazioneCritica() +
            c.getA3().getApprovazioneCritica() + c.getA4().getApprovazioneCritica())/4);
        c.setApprovazionePubblico((c.getA1().getApprovazionePubblico() + c.getA2().getApprovazionePubblico() +
            c.getA3().getApprovazionePubblico() + c.getA4().getApprovazionePubblico())/4);
        c.setIntesaRegista((c.getA1().getIntesaRegista() + c.getA2().getIntesaRegista() + c.getA3().getIntesaRegista()
            + c.getA4().getIntesaRegista())/4);
        c.setAffinitaGenere((c.getA1().getAffinitaGenere() + c.getA2().getAffinitaGenere() + c.getA3().getAffinitaGenere()
            + c.getA4().getAffinitaGenere())/4);

        c.setSintoniaAttori(normalization(c.getSintoniaAttori(), maxSintoniaAttori, minSintoniaAttori) *
            inputSintoniaAttori);
        c.setStatCast( c.getIncassoFilm() + c.getApprovazioneCritica() + c.getApprovazionePubblico() +
            c.getIntesaRegista() + c.getAffinitaGenere() + c.getSintoniaAttori());

        //controllo se è maxStatCast
        if(c.getStatCast() >= maxStatCast) {
            maxStatCast = c.getStatCast();
            bestCast.setA1(c.getA1());
            bestCast.setA2(c.getA2());
            bestCast.setA3(c.getA3());
            bestCast.setA4(c.getA4());
            bestCast.setIncassoFilm(c.getIncassoFilm());
            bestCast.setApprovazioneCritica(c.getApprovazioneCritica());
            bestCast.setApprovazionePubblico(c.getApprovazionePubblico());
            bestCast.setIntesaRegista(c.getIntesaRegista());
            bestCast.setAffinitaGenere(c.getAffinitaGenere());
            bestCast.setSintoniaAttori(c.getSintoniaAttori());
            bestCast.setStatCast(c.getStatCast());
        }
    }
}
```

4.2.3 Algoritmi per mostrare o modificare l'output

Il cast viene mostrato in una **ListView<Actor>**, che permette all'utente di selezionare gli attori. Ad ogni selezione, partono la funzione **doSelectActor()**, che riempie l'area di testo con le informazioni dell'attore, e la tabella con i film a cui ha preso parte fra i 1000 analizzati dal software.

```
@FXML
void doSelectActor(MouseEvent event) {

    txtCurriculum.clear();
    String info = "";
    Actor a = txtBestCast.getSelectionModel().getSelectedItem();

    info = info + a.getName() + "\nAge: " + (LocalDate.now().getYear() - a.getBirthYear())
        + "\n" + a.getProfession().substring(0,1).toUpperCase() + a.getProfession().replace(", ", ", ").substring(1);
    txtCurriculum.setText(info);

    List<Movie> lm = this.model.getMoviesActor(a);
    ObservableList<Movie> obs = FXCollections.observableArrayList(lm);
    tblCurriculum.setItems(obs);
}
```

Infine è prevista la funzione **replace()**, che permette di sostituire un attore del cast. Il metodo non fa partire un'ulteriore ricorsione, ma tramite un ciclo for trova il quarto attore che insieme ai tre già selezionati forma il cast con valore di **statCast** massimo.

```
public Cast replace(Cast oldCast, Actor rimosso, double inputSintoniaAttori) {
    // TODO Auto-generated method stub
    Cast newCast;
    List<Actor> parziale = new LinkedList<>();

    if(!oldCast.getA1().equals(rimosso)) {
        parziale.add(oldCast.getA1());
    }
    if(!oldCast.getA2().equals(rimosso)) {
        parziale.add(oldCast.getA2());
    }
    if(!oldCast.getA3().equals(rimosso)) {
        parziale.add(oldCast.getA3());
    }
    if(!oldCast.getA4().equals(rimosso)) {
        parziale.add(oldCast.getA4());
    }

    removed.add(rimosso);
    newCast = new Cast(parziale.get(0), parziale.get(1), parziale.get(2), null, 0,0,0,0,0,0);
    maxStatCast = 0;

    for(Actor a : this.grafo.vertexSet()) {

        if(!a.equals(rimosso) && !newCast.getA1().equals(a) && !newCast.getA2().equals(a) && !newCast.getA3()
            .equals(a) && !removed.contains(a)) {

            newCast.setA4(a);
            newCast.setSintoniaAttori(0);

            calculateSintoniaAttori(newCast);
            calculateCastStat(newCast, inputSintoniaAttori);

            if(newCast.getStatCast() >= maxStatCast) {
                maxStatCast = newCast.getStatCast();
                bestCast = new Cast(newCast.getA1(), newCast.getA2(), newCast.getA3(), newCast.getA4(), newCast.
                    getIncassoFilm(), newCast.getApprovazioneCritica(), newCast.getApprovazionePubblico(),
                    newCast.getIntesaRegista(), newCast.getIntesaRegista(), newCast.getSintoniaAttori(),
                    newCast.getStatCast());
            }
        }
    }
    System.out.println("\nFine");
    return bestCast;
}
```

5 Interfaccia e video dimostrativo

5.1 Sezione Genera Cast

The screenshot shows the 'Movie's Cast Builder' application with the 'Genera Cast' tab selected. The interface includes a title bar, two tabs ('Genera Cast' and 'Best Cast'), and a main content area. The main area contains several input fields and sliders for selecting movie parameters. At the bottom, there are 'Genera Cast' and 'Reset' buttons.

Movie's Cast Builder

Genera Cast Best Cast

Seleziona un genere cinematografico: Seleziona

Seleziona un regista: Seleziona

Seleziona i parametri di interesse:

Incasso del film

Approvazione della critica

Approvazione del pubblico

Sintonia tra gli attori

Intesa con il regista

Affinità con il genere scelto

Genera Cast Reset

5.2 Sezione Best Cast

The screenshot shows the 'Movie's Cast Builder' application with the 'Best Cast' tab selected. The interface includes a title bar, two tabs ('Genera Cast' and 'Best Cast'), and a main content area. The main area is divided into two columns: 'Best Cast' and 'Curriculum attore'. Below these columns is a table titled 'Cronologia film attore' with columns for 'Titolo', 'Regista', 'Genere', and 'Anno d'uscita'. The table is currently empty, showing the message 'Nessun contenuto nella tabella'. At the bottom, there is a 'Vuoi sostituire un attore?' dropdown menu and a 'Sostituisci' button.

Movie's Cast Builder

Genera Cast Best Cast

Best Cast Curriculum attore

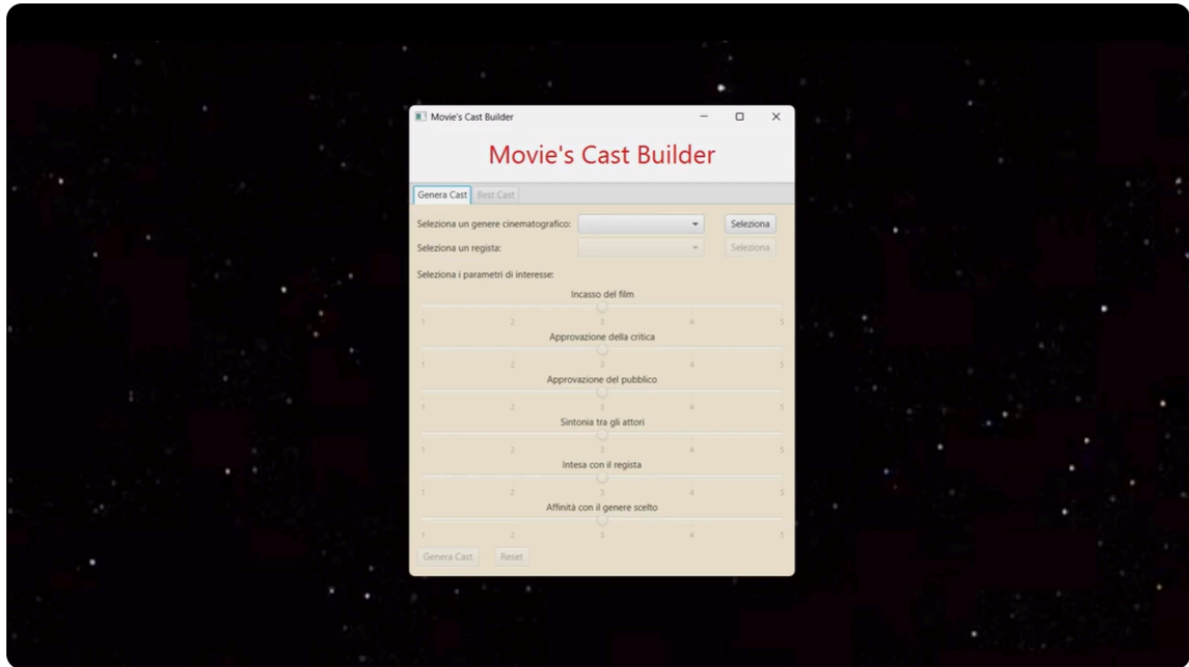
Cronologia film attore

Titolo	Regista	Genere	Anno d'uscita
Nessun contenuto nella tabella			

Vuoi sostituire un attore? Sostituisci

5.3 Video dimostrativo

<https://youtu.be/6GbIYA0LTdY>



Movie Cast Builder: individuare il miglior cast per una produzione cinematografica di successo



Diego Valle

Iscriviti



0



Condividi



Scarica



Salva



6 Risultati sperimentali e esempi

6.1 Esempio 1

The application 'Movie's Cast Builder' is shown in four states, illustrating how different parameter settings lead to different cast selections.

Top Left Screenshot: Parameters: Genere: Animation, Regista: Christopher Miller. Sliders: Incasso del film (4), Approvazione della critica (1), Approvazione del pubblico (1), Sintonia tra gli attori (3), Intesa con il regista (2), Affinità con il genere scelto (4). Result: Best Cast generated.

Top Right Screenshot: Parameters: Genere: Animation, Regista: Christopher Miller. Sliders: Incasso del film (4), Approvazione della critica (1), Approvazione del pubblico (1), Sintonia tra gli attori (3), Intesa con il regista (2), Affinità con il genere scelto (4). Result: Best Cast generated. Cast list: Joan Cusack, Lee Unkrich, Tim Allen, Tom Hanks. Curriculum: Tom Hanks, Age: 69, Producer, actor, writer.

Bottom Left Screenshot: Parameters: Genere: Animation, Regista: Christopher Miller. Sliders: Incasso del film (2), Approvazione della critica (1), Approvazione del pubblico (1), Sintonia tra gli attori (3), Intesa con il regista (2), Affinità con il genere scelto (4). Result: Best Cast generated.

Bottom Right Screenshot: Parameters: Genere: Animation, Regista: Christopher Miller. Sliders: Incasso del film (2), Approvazione della critica (1), Approvazione del pubblico (1), Sintonia tra gli attori (3), Intesa con il regista (2), Affinità con il genere scelto (4). Result: Best Cast generated. Cast list: Elizabeth Banks, Phil Lord, Rob Minkoff, Will Ferrell. Curriculum: Will Ferrell, Age: 58, Producer, actor, writer.

Titolo	Regista	Genere	Anno d'uscita
Forrest Gump	Robert Zemeckis	Drama, Romance	1994
Saving Private Ryan	Steven Spielberg	Drama, War	1998
The Green Mile	Frank Darabont	Crime, Drama, Fantasy	1999
Toy Story	John Lasseter	Animation, Adventure, C...	1995
Toy Story 3	Lee Unkrich	Animation, Adventure, C...	2010
Catch Me If You Can	Steven Spielberg	Biography, Crime, Drama	2002
Toy Story 2	John Lasseter	Animation, Adventure, C...	1999

Titolo	Regista	Genere	Anno d'uscita
The Lego Movie	Christopher Mil...	Animation, Action, Adven...	2014

Valutazione: Modificando i parametri di interesse si ottengono due cast completamente differenti. Ciò indica che, all'interno del gruppo di attori del genere selezionato, non esistono quattro interpreti nettamente superiori agli altri.

6.3 Esempio 2

The application 'Movie's Cast Builder' has two tabs: 'Genera Cast' and 'Best Cast'. The 'Best Cast' tab is active in all screenshots. The interface includes dropdowns for 'Seleziona un genere cinematografico:' (Crime) and 'Seleziona un regista:' (Todd Phillips), and a 'Seleziona' button. Below these are seven sliders for parameters: 'Incasso del film', 'Approvazione della critica', 'Approvazione del pubblico', 'Sintonia tra gli attori', 'Intesa con il regista', and 'Affinità con il genere scelto'. The 'Best Cast' list on the right shows Aaron Eckhart, Frances Conroy, John Travolta, and Zazie Beetz. The 'Curriculum attore' shows Aaron Eckhart's details. The 'Cronologia film attore' table shows 'The Dark Knight' by Christopher No...

Screenshot 1 (Top Left): Parameters: Incasso del film (2), Approvazione della critica (4), Approvazione del pubblico (5), Sintonia tra gli attori (2), Intesa con il regista (2), Affinità con il genere scelto (3). Best Cast: Aaron Eckhart, Frances Conroy, John Travolta, Zazie Beetz.

Screenshot 2 (Top Right): Parameters: Incasso del film (2), Approvazione della critica (4), Approvazione del pubblico (5), Sintonia tra gli attori (2), Intesa con il regista (2), Affinità con il genere scelto (3). Best Cast: Aaron Eckhart, Frances Conroy, John Travolta, Zazie Beetz.

Screenshot 3 (Bottom Left): Parameters: Incasso del film (4), Approvazione della critica (3), Approvazione del pubblico (1), Sintonia tra gli attori (3), Intesa con il regista (4), Affinità con il genere scelto (5). Best Cast: Aaron Eckhart, Frances Conroy, Robert De Niro, Zazie Beetz.

Screenshot 4 (Bottom Right): Parameters: Incasso del film (4), Approvazione della critica (3), Approvazione del pubblico (1), Sintonia tra gli attori (3), Intesa con il regista (4), Affinità con il genere scelto (5). Best Cast: Aaron Eckhart, Frances Conroy, Robert De Niro, Zazie Beetz.

Valutazione: Variando i parametri di interesse si ottengono due cast piuttosto simili. Questo suggerisce che, nella pool di attori del genere considerato, potrebbero esserci alcuni interpreti con punteggi significativamente più alti rispetto agli altri, oppure che le modifiche ai parametri non sono state abbastanza rilevanti da alterare il risultato.

6.2 Esempio 3

Movie's Cast Builder

Genera Cast | Best Cast

Seleziona un genere cinematografico: Fantasy Selezione

Seleziona un regista: Woody Allen Selezione

Seleziona i parametri di interesse:

- Incasso del film: 1 2 3 4 5
- Approvazione della critica: 1 2 3 4 5
- Approvazione del pubblico: 1 2 3 4 5
- Sintonia tra gli attori: 1 2 3 4 5
- Intesa con il regista: 1 2 3 4 5
- Affinità con il genere scelto: 1 2 3 4 5

Genera Cast Reset Best cast generato!

Movie's Cast Builder

Genera Cast | Best Cast

Best Cast

- Daniel Radcliffe
- Emma Watson
- Michelle Rodriguez
- Rupert Grint

Curriculum attore

Emma Watson
Age: 35
Actress, director, producer

Cronologia film attore

Titolo	Regista	Genere	Anno d'uscita
Harry Potter and the ...	David Yates	Adventure, Drama, Fantasy	2011
The Perks of Being a ...	Stephen Chbosky	Drama, Romance	2012
Harry Potter and the ...	Alfonso Cuarón	Adventure, Family, Fantasy	2004
Little Women	Greta Gerwig	Drama, Romance	2019
Harry Potter and the ...	David Yates	Adventure, Family, Fantasy	2010
Harry Potter and the ...	Mike Newell	Adventure, Family, Fantasy	2005
Harry Potter and the ...	David Yates	Action, Adventure, Family	2009

Vuoi sostituire un attore? Sostituisci

Valutazione: Mantenendo invariati i parametri di interesse, si ottiene il cast con il massimo potenziale di successo, senza influenze dovute a eventuali esigenze specifiche della casa di produzione

7 Valutazioni sui risultati ottenuti

L'output del codice appare solido e coerente. Sarebbe tuttavia utile disporre di un numero maggiore di film: alcuni titoli di grande successo, infatti, tendono a rendere la selezione di quattro attori all'interno di un determinato genere molto più probabile rispetto ad altri. Nonostante ciò, modificando in modo significativo i parametri di interesse, è possibile ottenere cast sempre differenti.

Il modello potrebbe essere ulteriormente migliorato includendo informazioni aggiuntive, come il cachet degli attori, dal momento che il budget rappresenta un fattore che una casa cinematografica non può trascurare.

Sarebbe inoltre vantaggioso disporre di un dataset completo che colleghi ciascun attore a tutti i film della propria carriera, così da evitare l'uso di euristiche nei casi di omonimia.

Infine, per rendere l'output più realistico, si potrebbe tenere conto anche dell'inflazione nel calcolo degli incassi dei film.



© 2025 Valle Diego

Movie Cast Builder: individuare il miglior cast per una produzione cinematografica di successo

è distribuito con licenza [Creative Commons Attribuzione - Non Commerciale - Condividi allo stesso modo 4.0 Internazionale \(CC BY-NC-SA 4.0\)](https://creativecommons.org/licenses/by-nc-sa/4.0/).